EE5203 L10 Worksheet

UART and PC communication - Basri Erdogan

Expected time: 30-45 minutes

Policy: Attempt every question before consulting the worked solutions.

`PCLK1` = 16 MHz, oversampling by 16, `8N1` framing throughout.

Part A - The frame and its cost

1. Draw an `8N1` frame for the byte `0x41` (`'A'`), labelling the start bit, each data bit in transmission order, and the stop bit. State the idle level.
2. Why does a byte cost 10 bit times rather than 8? What is the framing overhead as a percentage?
3. Complete the table.

Baud	µs per byte	bytes/s	20-char line (ms)
9 600			
38 400			
115 200			

4. A UART has no clock line. Explain how the receiver knows where each bit begins, and what it does at the start of every frame.
5. Two devices agree on baud, data bits and stop bits, but one uses even parity and the other none. Describe what the receiving side observes.

Part B - `BRR` arithmetic

6. Write the formula for USARTDIV, and explain how the result is stored in `BRR`.
7. Compute mantissa, fraction, `BRR` (hex), actual baud, and error for each:

Requested baud	Mantissa	Fraction	`BRR`	Actual	Error
9 600					
19 200					
115 200					
230 400					

8. A UART link tolerates roughly ±2 % total error across both ends. Which entry in your table would worry you, and why — given that the *other* device has its own error budget?
9. The board is moved to a configuration where `PCLK1` becomes 42 MHz. Compute `BRR` for 115 200 baud, and state whether the error improves or worsens.

Part C - Transmitting

10. Explain the difference between `TXE` and `TC`, and give one situation where only `TC` will do.
11. `HAL_UART_Transmit(&huart2, buf, 18, 100)` at 115 200 baud. How long does it block? Express that as a fraction of a 1 ms tick period, and state whether it belongs in a timer callback.
12. Why is `snprintf` preferred over `sprintf`? Describe precisely what goes wrong when a 60-character line is written into a `char buf[32]` with each.
13. Write the loop body that sends `<g_ms>,<raw>,<mv>` at exactly 5 Hz from the L07 tick, with CRLF endings and no blocking beyond the transmit itself.
14. A team retargets `printf` through `__io_putchar` and uses it for 10 Hz telemetry. Give two distinct reasons this is worse than `snprintf` plus one `HAL_UART_Transmit`.

Part D - Receiving

15. Name the flag that says a byte has arrived, and state what clears it.

16. Write the interrupt receive callback for USART2 that stores one byte into a ring buffer and re-arms. Mark the line that is most often forgotten.

17. Explain why blocking `HAL_UART_Receive` is unacceptable in this project. Name what stops working while it waits.

18. What is `ORE`, when does it set, and what happens to reception if it is never cleared? Give three plausible causes in order of likelihood.

19. Design a rule for assembling a command line from single bytes: state your terminator, your maximum length, and what you do when the maximum is exceeded. Explain why the bounds check cannot be omitted.

Part E - Diagnosis and evidence

20. For each symptom, give the most likely cause and the first thing you would check.

Symptom	Likely cause	First check
Nothing appears at all		
Consistent garbage characters		
Exactly one byte received, then silence		
Text stair-steps down the screen		
Everything freezes when you type		
Flashing fails with a port error		

21. Fill in the evidence you would show for each claim.

Claim	SFR observation
The baud divider is what I computed	
The pins belong to the USART	
A byte is waiting to be read	
Bytes were lost	

22. Your telemetry is 18 characters at 10 Hz on a 115 200 baud link. A teammate proposes raising the rate to 200 Hz "since the link is only 1.6 % used." Evaluate the proposal with numbers, and name the constraint that bites before bandwidth does.

Exit self-assessment

☐ I can compute `BRR` and the resulting error from memory.
☐ I can convert baud to line time and to lines per second.
☐ I can explain `TXE`/`TC` and `RXNE`/`ORE`.
☐ I can write an interrupt receive with a correct re-arm.
☐ I can diagnose the six standard UART symptoms.